

D4 - SYSTEM DESIGN DOCUMENT (SDD)

AgriUAV Platform: Precision Drone Dispatching Platform in the Mekong Delta

Course	ITA301 - Information System Analysis & Design
Deliverable	D4 – System design document (SDD)
Standard Applied	IEEE Std 830-1998
Group	Group 3
Members	• Dang Thanh Cong - Project Manager • Nguyen Thi Thanh Thao - Business Analyst • Le Van Minh - System Analyst • Tran Anh Khoa - System Designer • Dang Vo Thanh Hieu - Presenter / QA
Instructor	Mr. Tran Thanh Nguyen
Submission Date	2026/07/19
Version	v4.7

1. INTRODUCTION & CONTEXTUAL RATIONALE

This document serves as the System Design Document (SDD) for the AgriUAV Platform, translating the comprehensive software requirements specified in the Software Requirements Specification (SRS) into a robust, scalable, and verifiable technical blueprint. Grounded firmly in the Research-Based Learning (RBL) and Design Science Research (DSR) paradigms championed by FPT University, our architectural decisions transcend pure engineering abstractions and are rooted in empirical evidence derived from actual agricultural conditions within the Mekong Delta.

1.1 Evidence-Driven Architectural Constraints

1. Human-System Interaction Constraint: According to empirical research on human-system interaction in rural ecosystems (Nguyen 2025, N=940), the primary operators and agricultural cooperative workers have an average age of 51 with low base digital literacy. This severe human constraint requires that the Presentation Layer minimizes cognitive load through big typography, tactile button states, and simplified flows, while delegating heavy calculations, input parsing, and complex validation workflows entirely to automated asynchronous backend processes.

2. Network Infrastructure Vulnerability: Field testing in remote areas of Long An and Dong Thap reveals significant 4G network fragmentation and dropouts. This infrastructural reality forces the departure from cloud-only architectures toward a structured Offline-First synchronization protocol. Transactions cannot block or fail due to latency spikes or connection dropped events.

3. Stringent Regulatory Compliance: Regulatory policies outlined in Decree No. 288/2025/NĐ-CP require absolute traceability regarding flight paths, operator licenses, drone registrations, and exact pesticide chemical distribution logs. Data designs must provide absolute data integrity and non-repudiation chains that are easily auditable by inspectors from the Plant Protection Department.

2. UI DESIGN

2.1 User Personas

Based on the stakeholder analysis and the Role-Based Access Control requirement (FR18) defined in the SRS (D2), the AgriUAV platform serves five distinct user groups. The following personas outline their technological proficiency, primary goals, and specific UI/UX preferences. These profiles act as the foundation for our design decisions, ensuring the system caters to both low-tech users (Farmers) and specialized operators (Agronomists, Admins).

	Persona 1: Farmer	Persona 2: Operator/Pilot	Persona 3: HTX Manager	Persona 4: Agronomist	Persona 5: Admin
Character name	Mr. Tu (51 years old)	Mr. Ba (28 years old)	Mr. Nam (45 years old)	Ms. Hoa (35 years old)	Mr. Quan (30 years old)
Technology Proficiency	Low. Uses smartphones only for basic tasks.	Medium. Familiar with mobile applications and digital mapping.	Medium. Uses Excel, Zalo, and computers daily.	High. Proficient in Excel, mapping software, and GIS.	High. Experienced with system administration and software.
Goals	Schedule pesticide spraying in a very simple way. Receive notifications when tasks are completed.	Receive task assignments (task creation). Check in/out via GPS while working in the field to record work progress.	Monitor the entire drone fleet via an Overview Dashboard and Calendar; click to view details of each task.	Record and monitor detailed field indicators. Perform in-depth analyses. View historical records.	Manage user accounts, permissions, and system configurations (RBAC). Track audit logs and monitor security.
UI/UX Preferences	Always sets schedules with minimal steps (5 steps). Large font (>16pt), clear icons, direct interaction, voice support.	Large and clear buttons. Strong visual alerts (Alerts), visible even in outdoor light. Has Offline Mode.	Modern web interface. Desktop dashboard. Customizable widget cards. Master-detail layout. Supports Calendar view.	Simple web interface with Map module. Polygon-based field mapping with data input forms.	Admin panel web interface. Uses table-based layouts (data tables) with advanced filtering/search capabilities.
Primary Device Used	Mobile App (Android/iOS)	Mobile App (Android/iOS)	Web Dashboard (Desktop)	Web Dashboard (Desktop)	Web Dashboard (Desktop)

2.2 User Journey Map

To ensure a seamless and intuitive user experience, the team mapped out two critical user journeys that represent the core business workflows of the MVP: the Farmer's booking process (linked to UC01) and the Pilot's field operation process (linked to UC03 & UC05). These maps identify key touchpoints, potential pain points, and direct opportunities for UI/UX enhancements, particularly addressing field constraints like offline scenarios (NFR06).

Example: Farmer's Drone Spraying Booking Journey (Mapped with UC01 & UC08)

Phases	1. Awareness	2. Action	3. Confirmation	4. Post-Action
User actions	Opens the AgriUAV App, selects the field to be sprayed.	Chooses date/time, selects an approved pesticide (BVTV).	Receives booking confirmation from the HTX Manager.	Receives a Zalo message stating "Spraying completed" and views the report.
Touchpoints	"Field List" Screen (Home Screen).	"Booking" Screen (Booking Wizard).	Push Notification / Zalo.	"Flight History" Screen / Zalo Message.
Emotions/ Thoughts	"Hope this app is easy to use and I pick my correct field #2."	"Can I use this pesticide? How do I fill this out quickly?"	"Relieved! Someone will come to spray the field tomorrow."	"So convenient, I don't even have to go out to the field to monitor."
UI/UX Opportunities	Display the field area and a small map thumbnail so the farmer can easily recognize it.	The pesticide dropdown should only show valid options (isApprovedBVTV = true). Alert immediately if there is a scheduling conflict.	Use large, green text for the notification (Success) to provide a sense of reassurance.	Integrate a "Share" button via Zalo so they can easily forward the report to family members.

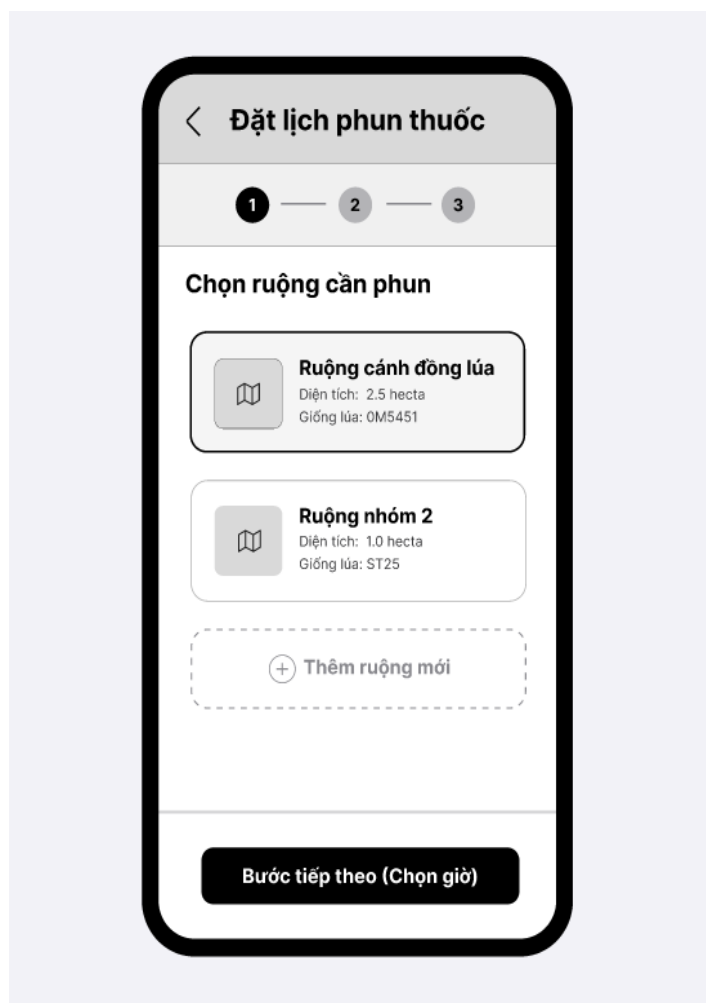
Example 2: Pilot's Flight Execution & Field Operations Journey (Mapped with UC03 & UC05)

Phases	1. Task Assignment	2. On-Site Check-In	3. Mission Execution	4. Mission Wrap-Up
User actions	Reviews assigned flight schedules, checks field coordinates, and views required pesticide formulas on the mobile app before leaving.	Arrives at the field, encounters poor 4G signal, opens the app to perform GPS check-in, and fills out the pre-flight checklist.	Launches the drone, monitors live telemetry data (battery, fluid levels) on the screen, and responds to boundary (geofence) alerts.	Lands the drone safely, submits the flight completion logs, and syncs data once back in network coverage.
Touchpoints	"My Schedule" Screen / Task Detail View.	"Field Check-In" Screen (with Offline Banner active).	Telemetry Dashboard / Map Overlay Screen.	"Mission Summary" Screen / Sync Progress Indicator.
Emotions/ Thoughts	"Let's check the field location and parameters. Looks like a standard spraying job."	"The 4G signal is completely dead out here. Glad the app still allows offline check-in so I can log my work."	"Keeping a close eye on the battery and the high-voltage power lines nearby. App warnings are clear."	"Mission accomplished safely. Uploading logs so the HTX Manager and Farmer can verify right away."
UI/UX Opportunities	Highlight prioritized or urgent tasks with clear time badges and large fonts.	Display a prominent "Offline Mode Enabled" banner to reassure the user. Use a giant, high-contrast GPS Check-In button for outdoor visibility under direct sunlight.	Implement distinct color-coded alerts (Yellow for caution, Red for danger) and optional audio/vibration feedback for critical geofence violations.	Create an automatic background data-syncing mechanism once cellular network is restored; show a clear sync status.

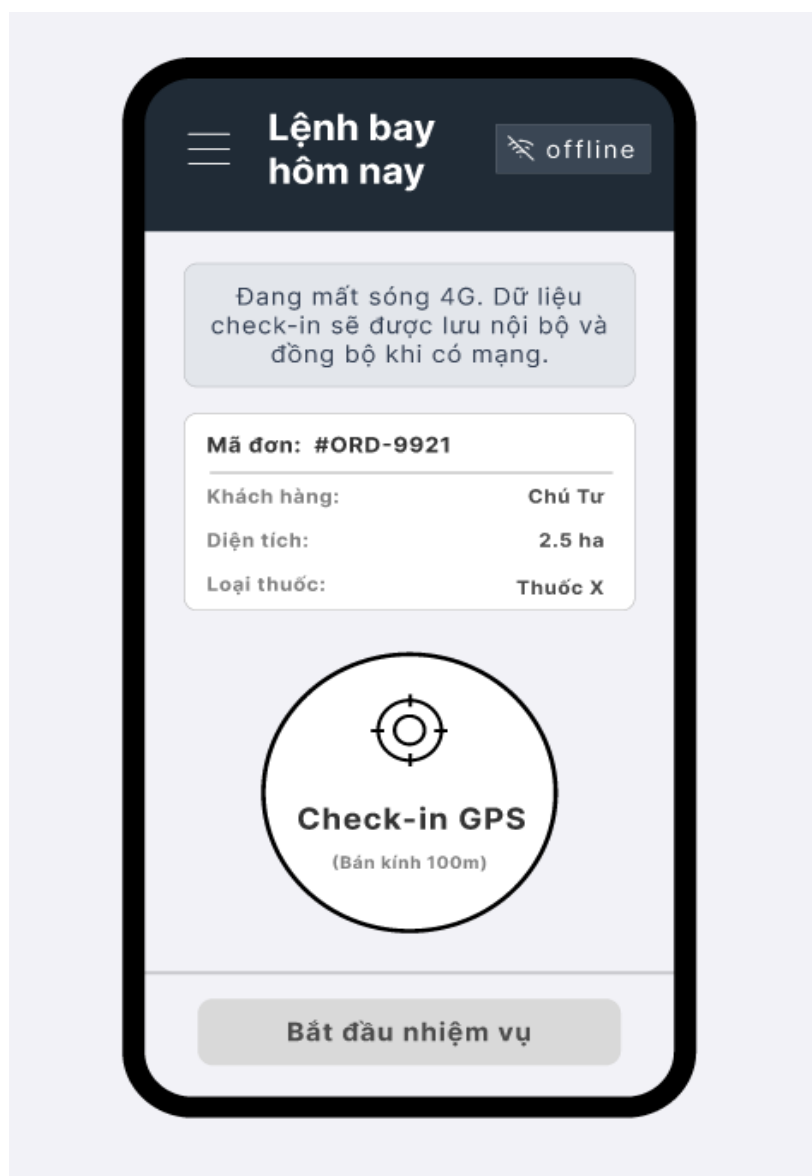
2.3 UI Wireframes

The following UI wireframes translate the defined personas and user journeys into structural screen layouts. Designed for both mobile (Farmer/Pilot) and web (HTX Manager) platforms, these wireframes incorporate critical non-functional requirements established in D2. Specifically, they feature large touch targets and text sizes ($\geq 16\text{pt}$) for accessibility (NFR03) and clear offline indicators for field operations (NFR06). The layouts intentionally focus on structure and flow rather than detailed visual aesthetics.

1. Booking (Farmer) – Mobile App



2. Check-in & Dashboard (Pilot) – Mobile App



3. Live Monitoring (HTX Manager) – Web

AgriUAV

Bản đồ (Live)

Đơn hàng phi công



K

Vừa ruộng A

DR-01

DR-02 (Cảnh báo)

Buffer 50m đường điện

Module Bản đồ GIS (Cập nhật 3s/lần)

Cảnh báo (1)

Danh sách drone

DR-02: Ranh giới

Vừa xong

Đang tiến sát vùng đệm 50m đường điện cao thế.

Xem camera

DR-01 (T40)

Đang bay

Phi công: Anh Ba

Pin: 65%

Thuốc: 12/40 L

Tốc độ: 5 m/s

4. Fleet Dispatch – Web

AgriUAV

Đơn hàng > Chờ phân công > #ORD-9921

Phân công nhiệm vụ bay

Chi tiết đơn

Khách hàng:

Chủ Tư (SĐT: 0901...)

Đồng ruộng:

Cánh Đồng Lớn (2.5 ha)

Thời gian yêu cầu:

08:00 AM - Ngày mai

Ghi chú từ AI (FR02):

08:00 AM - Ngày mai Cần khoảng 45 lít dung tích thuốc X. Đòi hỏi drone dung tích lớn.

Hệ thống đề xuất (Rule-based)

DJI Argas T40

Drone đề xuất 1

Mã: DR-001

Trạng thái:

Sẵn sàng

Pin:

100%

Dung tích:

40 Lít

Lý do: Dung tích đủ cover 90% ruộng trong 1 chuyến bay

Trần Văn Ba

Pilot đề xuất 2

Mã: PL-02

Chứng chỉ bay:

Hợp lệ (Còn 8th)

Lịch trình:

Trống 08:00 AM

Khoảng cách:

Cách ruộng 2km

Lý do: Ở gần nhất và có giấy phép hợp lệ (Decree 288)

Đổi tùy chọn thủ công

☒ Chốt phân công & Gửi thông báo

5. Spray Formula (Agronomist) – Web/ Mobile App

AgriUAV

Công thức phun

Bản đồ ruộng

Lịch sử

Tìm ruộng...

Ruộng cánh đồng lớn

Đã cấu hình

2.5 ha · Lúa OM5451

Chủ ruộng: Chú Tư

Ruộng Khóm 2

Chưa cấu hình

1.0 ha · Lúa ST25

Chủ ruộng: Chị Lan

Ruộng Bờ Kênh 5

Chưa cấu hình

3.2 ha · Lúa IR50404

Chủ ruộng: Anh Sáu

Công thức phun > Ruộng Cánh Đồng lớn

Cấu hình công thức phun

Ruộng Cánh Đồng Lớn · 2.5 ha · Lúa OM5451 · Chủ ruộng: Chú Tư

Sơ đồ ranh giới ruộng (polygon GPS)

Loại thuốc bảo vệ thực vật

Thuốc rầy X (Đã duyệt BVTV)

Danh sách chỉ hiển thị thuốc đã được Cục BVTV phê duyệt.

Liều lượng (lít/ha)

1.2

Số lần phun/vụ

3

Xem trước tổng lượng

Tổng lượng/lần (2.5 ha x 1.2 L):

3.0 lít

Cả vụ (x 3 lần phun):

9.0 lít

Người cấu hình: Ms. Hoa (Agronomist) · Cập nhật gần nhất: 10/07/2026

Hủy thay đổi

Lưu & Xác nhận

3. DATA DESIGN

The data tier for the AgriUAV Platform is architected to support transactional ACID guarantees alongside complex geospatial query capabilities. To achieve this, the system implements an Object-Relational schema utilizing PostgreSQL enhanced with the PostGIS spatial extension. This allows the system to evaluate geometries natively at the database engine level.

3.1 Physical Database Schema (PostgreSQL DDL Script)

The following physical database schema translates the AgriUAV system data models into concrete relational definitions. The data architecture is built upon PostgreSQL 16 as the core database engine, strategically integrated with two key extensions:

- o **PostGIS 3.4:** For handling complex spatial queries and geographic farm mapping.
- o **TimescaleDB 2.x:** For managing high-frequency, time-series drone telemetry data.

To ensure distributed consistency and scalability, all primary keys across the system utilize UUID v4. Additionally, in alignment with regulatory standards (Decree 288/2025/ND-CP), critical historical data including `flight_logs`, `drone_positions`, and `audit_logs` are designed with a strict minimum retention period of 5 years.

1. USERS (System Users)

```
CREATE TABLE users (  
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
    email VARCHAR(255) UNIQUE NOT NULL,  
    password_hash VARCHAR(255) NOT NULL,  
    full_name VARCHAR(100) NOT NULL,  
    role VARCHAR(50) NOT NULL CHECK (role IN ('admin','htx_manager','operator','farmer','agronomist')),  
    phone_number VARCHAR(15),  
    is_active BOOLEAN NOT NULL DEFAULT TRUE,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
COMMENT ON TABLE users IS 'Stores general user information across the system';  
  
COMMENT ON COLUMN users.is_active IS 'Soft-delete flag. Account deactivation must use this flag rather than hard  
DELETE, preserving referential integrity for audit_logs.actor_id (ON DELETE RESTRICT) and Decree 288/2025/ND-  
CP traceability.';
```

2. OPERATORS (Pilot Profiles)

```
|CREATE TABLE operators (|
| id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),|
| user_id UUID NOT NULL UNIQUE,|
| certificate_number VARCHAR(100) UNIQUE NOT NULL,|
| certificate_expiry DATE NOT NULL,|
| experience_years INT DEFAULT 0,|
| status VARCHAR(50) DEFAULT 'ACTIVE' CHECK (status IN ('ACTIVE', 'ON_LEAVE', 'SUSPENDED')),|
| CONSTRAINT fk_operator_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE|
|);|
|COMMENT ON TABLE operators IS 'Stores professional information and certifications of operators/pilots';|
```

3. COOPERATIVES

```
|CREATE TABLE cooperatives (|
| id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),|
| name VARCHAR(255) NOT NULL,|
| province VARCHAR(100) NOT NULL,|
| address TEXT,|
| max_drones INTEGER NOT NULL DEFAULT 50,|
| manager_id UUID NOT NULL,|
| created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,|
| CONSTRAINT fk_cooperative_manager FOREIGN KEY (manager_id) REFERENCES users(id) ON DELETE|
|RESTRICT|
|);|
|COMMENT ON TABLE cooperatives IS 'HTX entity per D3 §2.1; required for drones.cooperative_id FK.';|
```

4. FIELDS (Farming Areas / Plots)

```

CREATE TABLE fields (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  farmer_id UUID NOT NULL,
  cooperative_id UUID NOT NULL,
  name VARCHAR(255) NOT NULL,
  crop_type VARCHAR(100) NOT NULL,
  area_size DECIMAL(10,2) NOT NULL,
  location_polygon GEOMETRY(Polygon, 4326),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_field_farmer FOREIGN KEY (farmer_id) REFERENCES users(id) ON DELETE CASCADE,
  CONSTRAINT fk_field_cooperative FOREIGN KEY (cooperative_id) REFERENCES cooperatives(id) ON DELETE
  RESTRICT
);

COMMENT ON TABLE fields IS 'Stores information about farmers" farming fields';

COMMENT ON COLUMN fields.crop_type IS 'e.g. rice, corn — matches D3 fields.cropType (D3 §3)';

```

5. GEOFENCES

```

CREATE TABLE geofences (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  field_id UUID NOT NULL UNIQUE,
  boundary_polygon GEOMETRY(Polygon, 4326) NOT NULL,
  buffer_meters INTEGER NOT NULL DEFAULT 50,
  hazard_zones JSONB,
  is_active BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_geofence_field FOREIGN KEY (field_id) REFERENCES fields(id) ON DELETE CASCADE
);

COMMENT ON TABLE geofences IS 'Safety boundary per field (1:1 via UNIQUE field_id, D3 §2.2/§3). Alert behaviour
is information-level only per FR12 — the system never directly controls the drone, firmware, or autopilot; the pilot
remains responsible for flight safety.';

```

COMMENT ON COLUMN geofences.buffer_meters IS 'Default 50m buffer around registered high-voltage power lines, informed by the Long An 110kV incident (Tuoi Tre, 2024; D2 §1.4). Configurable per field.';

COMMENT ON COLUMN geofences.hazard_zones IS 'JSONB array, e.g. {type: POWERLINE|RESIDENTIAL, coords} (D3 §3)';

COMMENT ON COLUMN geofences.is_active IS 'Set TRUE on UC03 check-in — geofence monitoring activates only once the Pilot is on-site (D3 §3, UC03 Postconditions)';

6. PESTICIDES (Pesticide Catalog)

```
CREATE TABLE pesticides (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  name VARCHAR(255) NOT NULL,  
  active_ingredient VARCHAR(255) NOT NULL,  
  is_approved_bvtv BOOLEAN NOT NULL DEFAULT FALSE,  
  unit VARCHAR(20) NOT NULL DEFAULT 'L',  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

COMMENT ON TABLE pesticides IS 'BVTv-approved master registry (D3 §2.1/§3 exact match). is_approved_bvtv checked at booking (UC01 Alt 4a) and order creation (FR03).';

7. PESTICIDE STOCK

```
CREATE TABLE pesticide_stock (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  cooperative_id UUID NOT NULL,  
  pesticide_id UUID NOT NULL,  
  stock_l DECIMAL(8,2) NOT NULL DEFAULT 0,  
  low_stock_threshold_l DECIMAL(8,2) NOT NULL,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  CONSTRAINT fk_stock_cooperative FOREIGN KEY (cooperative_id) REFERENCES cooperatives(id) ON DELETE CASCADE,  
  CONSTRAINT fk_stock_pesticide FOREIGN KEY (pesticide_id) REFERENCES pesticides(id) ON DELETE RESTRICT,
```

```

| CONSTRAINT uq_stock_cooperative_pesticide UNIQUE (cooperative_id, pesticide_id)
|
|);
|
|COMMENT ON TABLE pesticide_stock IS 'Per-cooperative pesticide inventory, split from the pesticides master
|registry so stock can vary across cooperatives sharing the same approved list (D3 §2.1/§3). Deducted after a
|COMPLETED spraying mission (FR16).';
|
|COMMENT ON COLUMN pesticide_stock.low_stock_threshold_I IS 'Low-stock alert fires when stock_I <
|low_stock_threshold_I (FR16)';

```

8. SPRAY_FORMULAS

```

|CREATE TABLE spray_formulas (
|
| id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
|
| field_id UUID NOT NULL UNIQUE,
|
| pesticide_id UUID NOT NULL,
|
| dosage_l_per_ha DECIMAL(6,2) NOT NULL,
|
| spray_count_per_season SMALLINT NOT NULL,
|
| configured_by UUID NOT NULL,
|
| created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
|
| CONSTRAINT fk_formula_field FOREIGN KEY (field_id) REFERENCES fields(id) ON DELETE CASCADE,
|
| CONSTRAINT fk_formula_pesticide FOREIGN KEY (pesticide_id) REFERENCES pesticides(id) ON DELETE
|RESTRICT,
|
| CONSTRAINT fk_formula_configured_by FOREIGN KEY (configured_by) REFERENCES users(id) ON DELETE
|RESTRICT
|);
|
|COMMENT ON TABLE spray_formulas IS 'Per-field spray formula configured by the Agronomist (FR02); reused
|across all spray_orders for that field. UNIQUE(field_id) enforces the 1:1 relationship with fields, matching D3 §2.1/§3.';

```

9. DRONES (UAV Equipment)

```

|CREATE TABLE drones (
|
| id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
|
| cooperative_id UUID NOT NULL,
|
| serial_number VARCHAR(100) UNIQUE NOT NULL,
|
| model VARCHAR(100) NOT NULL,
|
| tank_capacity DECIMAL(10,2) NOT NULL,

```

```

| status VARCHAR(50) DEFAULT 'IDLE' CHECK (status IN ('IDLE', 'FLYING', 'MAINTENANCE', 'OFFLINE')),
| battery_percent SMALLINT DEFAULT 100,
| gps_lat DECIMAL(9,6),
| gps_lng DECIMAL(9,6),
| created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
| updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
| CONSTRAINT fk_drone_cooperative FOREIGN KEY (cooperative_id) REFERENCES cooperatives(id) ON
DELETE RESTRICT
|);
|
|COMMENT ON TABLE drones IS 'Manages the list of Drones in the system';

```

10. DRONE_MAINTENANCE (Drone Maintenance Logs)

```

|CREATE TABLE drone_maintenance (
|
| id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
| drone_id UUID NOT NULL,
| maintenance_date DATE NOT NULL,
| description TEXT NOT NULL,
| cost DECIMAL(12,2) DEFAULT 0,
| performed_by VARCHAR(100),
|
| CONSTRAINT fk_maintenance_drone FOREIGN KEY (drone_id) REFERENCES drones(id) ON DELETE
CASCADE
|);
|
|COMMENT ON TABLE drone_maintenance IS 'Tracks the maintenance and repair history of flight equipment';

```

11. SPRAY_ORDERS (Spray Scheduling Orders)

```

|CREATE TABLE spray_orders (
|
| id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
| farmer_id UUID NOT NULL,
| field_id UUID NOT NULL,
| drone_id UUID,

```

```

| operator_id UUID,
|
| scheduled_date TIMESTAMP NOT NULL,
|
| actual_start_time TIMESTAMP,
|
| status VARCHAR(50) NOT NULL DEFAULT 'PENDING' CHECK (status IN ('PENDING', 'ASSIGNED',
| 'IN_PROGRESS', 'COMPLETED', 'SUSPENDED', 'CANCELLED')),
|
| status_reason TEXT,
|
| cancel_reason TEXT,
|
| created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
|
| CONSTRAINT fk_order_farmer FOREIGN KEY (farmer_id) REFERENCES users(id) ON DELETE CASCADE,
|
| CONSTRAINT fk_order_field FOREIGN KEY (field_id) REFERENCES fields(id) ON DELETE CASCADE,
|
| CONSTRAINT fk_order_drone FOREIGN KEY (drone_id) REFERENCES drones(id) ON DELETE SET NULL,
|
| CONSTRAINT fk_order_operator FOREIGN KEY (operator_id) REFERENCES operators(id) ON DELETE SET
| NULL
|
|);
|
| COMMENT ON TABLE spray_orders IS 'Central spray scheduling orders of the system';

```

12. FLIGHT_LOGS (Flight Logs)

```

| CREATE TABLE flight_logs (
|
| id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
|
| order_id UUID NOT NULL,
|
| drone_id UUID NOT NULL,
|
| operator_id UUID NOT NULL,
|
| start_time TIMESTAMP,
|
| end_time TIMESTAMP,
|
| check_in_lat DECIMAL(9,6),
|
| check_in_lng DECIMAL(9,6),
|
| total_area_sprayed DECIMAL(10,2),
|
| pesticide_used_l DECIMAL(6,2),
|
| gps_trail JSONB,
|
| manual_override BOOLEAN NOT NULL DEFAULT FALSE,

```

```

| synced_at TIMESTAMP,
| status VARCHAR(50) DEFAULT 'IN_FLIGHT' CHECK (status IN ('IN_FLIGHT', 'COMPLETED', 'FAILED')),
| CONSTRAINT uq_flight_order UNIQUE (order_id),
| CONSTRAINT fk_flight_order FOREIGN KEY (order_id) REFERENCES spray_orders(id) ON DELETE CASCADE,
| CONSTRAINT fk_flight_drone FOREIGN KEY (drone_id) REFERENCES drones(id) ON DELETE RESTRICT,
| CONSTRAINT fk_flight_operator FOREIGN KEY (operator_id) REFERENCES operators(id) ON DELETE
| RESTRICT
|);

| COMMENT ON TABLE flight_logs IS 'Records overview information of each takeoff/flight; drone_id gives a direct
| compliance-chain edge without joining through spray_orders (per D3 §2.1)';

| COMMENT ON COLUMN flight_logs.pesticide_used_l IS 'Actual pesticide volume entered by Pilot at check-out
| (UC07 step 2); compared against spray_formulas.dosage_l_per_ha × fields.area_size to trigger UC07 Alt 2a (>20%
| deviation)';

```

13. DRONE_POSITIONS (Telemetry Data / Flight Coordinates)

```

| CREATE TABLE drone_positions (
|
| id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
|
| flight_log_id UUID NOT NULL,
|
| latitude DECIMAL(10,8) NOT NULL,
|
| longitude DECIMAL(11,8) NOT NULL,
|
| altitude DECIMAL(10,2) NOT NULL,
|
| speed DECIMAL(10,2),
|
| recorded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
|
| CONSTRAINT fk_position_flight FOREIGN KEY (flight_log_id) REFERENCES flight_logs(id) ON DELETE
| CASCADE
|);

| COMMENT ON TABLE drone_positions IS 'Real-time coordinate data of Drones (used for redrawing flight paths)';

```

14. WEATHER_CONDITIONS (Weather Logs)

```

| CREATE TABLE weather_conditions (
|
| id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
|

```

```

| order_id UUID NOT NULL,
| temperature DECIMAL(5,2),
| wind_speed DECIMAL(5,2),
| humidity DECIMAL(5,2),
| recorded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
| CONSTRAINT fk_weather_order FOREIGN KEY (order_id) REFERENCES spray_orders(id) ON DELETE
| CASCADE
|);
|COMMENT ON TABLE weather_conditions IS 'Stores weather conditions at the time of flight for safety reference';

```

15. REPORTS (Legal/Compliance Reports)

```

|CREATE TABLE reports (
| id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
| generated_by UUID NOT NULL,
| cooperative_id UUID NOT NULL,
| report_type VARCHAR(50) NOT NULL CHECK (report_type IN ('DECREE_288', 'VIETGAP', 'GLOBALGAP')),
| period_start DATE NOT NULL,
| period_end DATE NOT NULL,
| file_format VARCHAR(10) NOT NULL CHECK (file_format IN ('PDF', 'XLSX')),
| file_url VARCHAR(255), -- Link to PDF/Excel file in Storage
| exported_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
| CONSTRAINT fk_report_user FOREIGN KEY (generated_by) REFERENCES users(id) ON DELETE CASCADE,
| CONSTRAINT fk_report_cooperative FOREIGN KEY (cooperative_id) REFERENCES cooperatives(id) ON
| DELETE RESTRICT
|);
|COMMENT ON TABLE reports IS 'Compliance-support export record (D3 §3 exact match). DECREE_288 = FR14
| (Plant Protection Dept report, layout pending authority confirmation); VIETGAP/GLOBALGAP = FR15 (project-defined
| minimum field set, pending export-company validation)';

```

16. NOTIFICATIONS (System Notifications)

```

|CREATE TABLE notifications (

```

```

| id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
|
| user_id UUID NOT NULL,
|
| title VARCHAR(255) NOT NULL,
|
| content TEXT NOT NULL,
|
| is_read BOOLEAN DEFAULT FALSE,
|
| created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
|
| CONSTRAINT fk_notification_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
|);
|
|COMMENT ON TABLE notifications IS 'In-app notification inbox for users';
|

```

17. AUDIT_LOGS (System/Security Logs)

```

|CREATE TABLE audit_logs (
|
| id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
|
| event_type VARCHAR(50) NOT NULL CHECK (event_type IN (
|
|   'ORDER_CREATED', 'ORDER_CANCELLED', 'WEATHER_OVERRIDE',
|
|   'REPORT_EXPORTED', 'DATA_ACCESS', 'CHECKIN', 'CHECKOUT'
|
|  )),
|
| actor_id UUID NOT NULL,
|
| actor_role VARCHAR(50) NOT NULL CHECK (actor_role IN (
|
|   'admin', 'htx_manager', 'operator', 'farmer', 'agronomist'
|
|  )),
|
| target_entity_id UUID NOT NULL,
|
| target_entity_type VARCHAR(50) NOT NULL,
|
| description TEXT NOT NULL,
|
| ip_address VARCHAR(45),
|
| occurred_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
|
| CONSTRAINT fk_audit_actor FOREIGN KEY (actor_id) REFERENCES users(id) ON DELETE RESTRICT
|);
|

```

```
COMMENT ON TABLE audit_logs IS 'Append-only audit trail (D3 §3 exact match). Covers weather overrides (UC01/UC06), report exports (UC08), data-sharing access approvals (UC10), and GPS check-in overrides (UC03 Alt 3a, event_type = CHECKIN). Supports NFR05"s immutable-audit-log outcome.';

COMMENT ON COLUMN audit_logs.actor_id IS 'ON DELETE RESTRICT (not SET NULL as the old system_logs had): a user with audit history cannot be hard-deleted, preserving Decree 288/2025/ND-CP traceability. Account deactivation should use a soft-delete flag on users instead.';

|-- Append-only enforcement: block UPDATE and DELETE at the DB level

CREATE OR REPLACE FUNCTION prevent_audit_log_modification()

RETURNS TRIGGER AS $$

BEGIN

    RAISE EXCEPTION 'audit_logs is append-only: % is not permitted (Decree 288/2025/ND-CP compliance requirement)', TG_OP;

END;

$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_audit_logs_append_only

BEFORE UPDATE OR DELETE ON audit_logs

FOR EACH ROW EXECUTE FUNCTION prevent_audit_log_modification();

CREATE INDEX idx_audit_actor_occurred ON audit_logs (actor_id, occurred_at DESC);
```

3.2 Indexing Strategy

To ensure the AgriUAV platform meets the strict real-time telemetry and spatial query requirements defined in the NFRs (e.g., latency < 500ms for system updates), the database relies on a robust indexing strategy. By implementing B-Tree and GiST (Generalized Search Tree) indexes, the system structurally reduces the search time complexity from $O(N)$ (Full Table Scan) to $O(\log N)$. This mathematical optimization is crucial for rapid relational joins across millions of flight logs and real-time geographic queries via PostGIS.

Table Name	Index Name	Column(s) Indexed	Index Type	Architectural & RBL Justification
fields	idx_fields_location	location_polygon	GiST	Arch: Reduces geographic spatial intersections to $O(\log N)$. RBL: Crucial for the Mekong Delta context where mapping complex, non-rectangular farming plots accurately is required for precision spraying.

Table Name	Index Name	Column(s) Indexed	Index Type	Architectural & RBL Justification
spray_orders	idx_orders_farmer_id	farmer_id	B-Tree	Arch: Accelerates relational JOINS between users and their orders. RBL: Ensures the mobile app loads historical data quickly even on unstable 3G networks in rural areas (Supporting NFR06 - Offline/Low Connectivity).
spray_orders	idx_orders_status	status	B-Tree	Arch: Prevents full table scans when filtering active missions. RBL: Satisfies the scalability threshold (NFR04) allowing the Cooperative (HTX) Manager to concurrently monitor 50+ drones without UI freezing.
flight_logs	idx_flights_order_id	order_id	B-Tree	Arch: Optimizes referential integrity checks. RBL: Directly supports regulatory compliance (Decree 288/2025/ND-CP) by instantly bridging executed flight telemetry with authorized spray orders for government auditing.
drone_positions	idx_positions_flight_id	flight_log_id	B-Tree	Arch: Handles high-throughput relational lookups for telemetry data. RBL: Essential to meet the strict real-time monitoring threshold (NFR01: Latency < 500ms) for fleet tracking and collision avoidance.
drone_positions	idx_positions_recorded	recorded_at	B-Tree	Arch: Optimizes time-series sorting (ORDER BY recorded_at DESC). RBL: Aligns with the architectural choice of TimescaleDB, enabling the system to fetch the absolute latest GPS ping of a drone instantly.
notifications	idx_notifications_unr	user_id, is_read	B-Tree (Composite)	Arch: Composite

Table Name	Index Name	Column(s) Indexed	Index Type	Architectural & RBL Justification
	ead			index minimizes I/O reads for polling unread alerts. RBL: Ensures operators receive critical weather or hardware warnings instantly, addressing safety requirements.

4. ARCHITECTURE DESIGN

To build a system capable of handling continuous telemetry ingestion while remaining robust against infrastructure constraints, we evaluated two architectural paradigms against the practical environment of regional farming cooperatives.

4.1 Architectural Paradigms Evaluated

Option 1: Pure Event-Driven Architecture (EDA)

- **Description:** This paradigm completely decentralizes the system. Every action (e.g., a drone taking off, a pilot checking in, a telemetry packet arriving) emits an event to a central message broker (such as Apache Kafka). Services react to these events asynchronously without direct API calls.
- **Assessment:** While highly efficient for streaming real-time telemetry, EDA relies entirely on Eventual Consistency. This poses a severe risk for core business logic, such as preventing double-booking of a drone during peak harvest season. Tracking down system failures (observability) in a pure EDA setup is also notoriously difficult for small technical teams.

Option 2: Distributed Microservices Architecture

- **Description:** Decomposes the application into strictly isolated, independent microservices (e.g., Scheduling Service, Telemetry Ingestion Service, Compliance Service), each managing its own separate database cluster. Communication occurs via synchronous APIs (REST/gRPC) or lightweight message queues.
- **Assessment:** This architecture offers extreme horizontal scalability and fault isolation. However, it introduces immense operational complexity. It demands active Kubernetes clusters, service meshes, and distributed Saga patterns to handle transactions across databases. This overhead drastically exceeds the technical and financial capacities of standard agricultural cooperatives.

Option 3: Layered Modular Monolith with Redis Buffer (**Chosen Paradigm -- Final Choice**)

- **Description:** Organizes code into structurally isolated modules (Fleet, Mapping, Billing, Compliance) inside a single deployable pipeline. High-frequency telemetry streams are intercepted by an in-memory Redis Stack Queue acting as a volatile storage buffer, which sequentially drains into a central PostgreSQL database via an asynchronous Write-Behind pattern.

Why This is Our Final Choice: This architecture perfectly balances performance with operational reality. It grants the strict ACID transactional guarantees required to prevent scheduling conflicts (via PostgreSQL), comfortably handles the required 200 req/s telemetry load (via the Redis buffer), and eliminates the crushing infrastructural overhead of managing distributed microservices. It is highly resilient, cost-effective, and deeply aligned with the Research-Based Learning (RBL) constraints regarding the limited technical personnel at local cooperatives.

4.2 Trade-off Evaluation Matrix

Evaluation Criteria (Mapped to NFRs)	Option 1: Pure Event-Driven	Option 2: Mircroservices	Option 3: Modular Monolith (Chosen)	Evidence-Based Justification (RBL Delta)
Data Consistency	Eventual Consistency. High risk of race conditions during booking.	Eventual/Saga Consistency. Complex to enforce absolute data integrity.	Strong ACID Compliance. Immediate row locking on core database tables.	Critical for fulfilling UC02 (Schedule Drone Spraying) . In the context of agricultural hardware, overlapping schedules or double-booking a drone instance can lead to severe operational conflicts. The chosen centralized database architecture leverages strict ACID transactions to instantly block concurrent booking attempts at the transaction boundary. This natively ensures data integrity without the overhead of complex distributed locks required in microservices..
Operational Complexity	High. Requires complex event tracing and dead-letter queue management.	Very High. Demands orchestration (Kubernetes) and dedicated DevOps.	Low to Moderate. Single compilation artifact, straightforward local backup procedures.	Empirical observations in the Mekong Delta reveal that Agricultural Cooperatives (HTX) completely lack dedicated DevOps teams or technical platform operators. The chosen Layered Architecture guarantees operational simplicity (single codebase, straightforward deployment). This simplicity minimizes potential downtime, strictly satisfying the NFR07 (Maintainability) constraint—which mandates that emergency hotfixes must be resolvable within \$\\le\$ 4 hours—and significantly lowers the overall Total Cost of Ownership (TCO) for local farmers.
Peak Throughput (200 req/s baseline)	Extremely High. Handles streaming effortlessly.	Extremely High. Scales horizontally past 10,000 req/s via container replication.	High. Comfortably achieves 1,500 req/s due to Redis write decoupling.	The evaluation workload baseline established in D2 requires the system to handle a telemetry peak of 200 requests per second (accounting for a maximum of

Evaluation Criteria (Mapped to NFRs)	Option 1: Pure Event-Driven	Option 2: Mircroservices	Option 3: Modular Monolith (Chosen)	Evidence-Based Justification (RBL Delta)
				50 active drones per HTX). The chosen architectural approach meets this exact quantitative threshold defined in NFR04 (Scalability) seamlessly. By avoiding over-engineering, the system handles the required scalability without the exponential infrastructure costs associated with container orchestration clusters (e.g., Kubernetes).
Offline Resilience	Complex. High risk of event duplication when networks reconnect.	Complex. Requires distributed sagas to reconcile delayed mobile uploads.	Streamlined. Ingests local arrays via a single state synchronization gateway.	Network instability is a proven reality in remote farming areas. The selected monolithic/layered backend vastly simplifies error recovery routines on Pilot mobile clients during UC03 (Pilot Check-in/out) . When 4G signals drop and subsequently restore, reconciling local offline sync data back into a single system core prevents eventual-consistency conflicts. This strongly supports NFR06 (Availability & Offline Mode) , ensuring that offline field data is flawlessly synchronized within the ≤ 30 seconds threshold upon network reconnection.

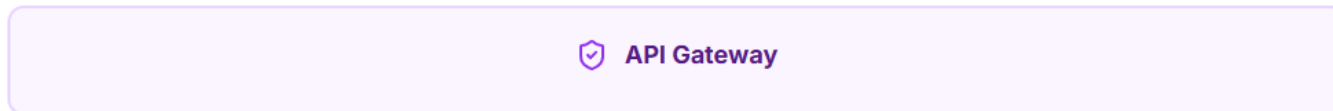
4.3 Final Architectural Choice + System Architecture Blueprint

The AgriUAV platform adopts a **Layered Architecture** pattern, specifically optimized for the operational constraints of agricultural cooperatives in the Mekong Delta (e.g., intermittent internet connectivity, limited IT personnel). This approach ensures rapid deployment, simplified maintenance (NFR07), and robust data consistency. The diagram below illustrates the logical separation of concerns across the Client, API, Application, and Data layers.

1. PRESENTATION LAYER (CLIENT)



2. SECURITY & GATEWAY LAYER



3. MODULAR APPLICATION CORE



4. SHARED INFRASTRUCTURE & DATA



Architectural Layer Breakdown

- **Client Layer:** Comprises specialized interfaces tailored for distinct operational personas:
 - Farmer Web/App: Optimized for usability and accessibility (NFR03).
 - Pilot App: Incorporates a local Offline SQLite database to ensure continuous operation during 4G network dropouts in remote fields (NFR06).
 - HTX Dashboard: A web-based interface for managers to monitor fleet status and schedules in real-time.
 - Drone: The physical IoT hardware transmitting telemetry and status data.
- **API Gateway Layer:** Acts as the unified entry point for all incoming client requests. It offloads cross-cutting concerns such as routing, rate-limiting (configured per drone device hardware ID to prevent rogue telemetry flooding from malfunctioning sensors), TLS termination; JWT validation is enforced downstream in the Application Layer.
- **Application Layer:** The centralized monolithic engine housing specific business modules:

- Booking Module: Manages strict transactional workflows for scheduling spraying sessions (UC01, UC02).
- Fleet Module: Handles drone dispatching, capacity planning, and status tracking.
- Telemetry Module: Designed to process high-throughput, real-time flight data originating from the drones (UC04).

● **Data Persistence Layer:** A hybrid, purpose-built storage environment:

PostgreSQL & PostGIS: Manages core relational data (users, bookings) and complex spatial geographic queries (farm mapping). TimescaleDB: An extension specifically chosen to handle time-series drone telemetry logs. Redis: Functions as an in-memory buffer to absorb high-frequency telemetry data streams before they are permanently flushed to the database, preventing database bottlenecks.

5. TECH STACK PROPOSAL

The component selections across every layer of the architecture have been specifically optimized to handle the demographic realities of rural operators (Nguyen 2025) and the infrastructure of agricultural environments:

- **Web Administration Dashboard (Frontend):** Built with [React.js](#) styled via [TailwindCSS](#). Map layouts are rendered using [Leaflet.js](#) integrating [OpenStreetMap](#) data layers. This provides full GIS capabilities without the commercial API licensing costs associated with Google Maps, aligning with cooperative budgetary limits.
- **Mobile Application (Pilots & Farmers) (Frontend):** Developed as a native cross-platform application using [Flutter \(Dart\)](#). Local data persistence uses an encrypted [SQLite](#) instance embedded directly in the client container. This architecture natively drives the Offline-First workflow [NFR06], storing telemetry records securely when 4G connectivity drops. To support the older farmer demographic [NFR03], the UI typography targets a minimum size of 16pt alongside high-contrast visual accessibility patterns.
- **Application Logic Layer (Backend):** Engineered on [NestJS \(Node.js/TypeScript\)](#). Leveraging Node's asynchronous, non-blocking event-loop model allows the backend gateway to sustain high-throughput [WebSockets Secure \(WSS\)](#) streaming from multiple operating drones simultaneously without thread exhaustion, fulfilling real-time monitoring constraints [NFR01].
Database & Volatile Caching Layer: PostgreSQL 16 serves as the core relational engine enforcing strict ACID transactional integrity for scheduling [UC02].
 - **The PostGIS 3.4** extension evaluates geographic primitives natively (e.g., `ST_Contains`) for field mapping.
 - **TimescaleDB 2.x** is tightly integrated to explicitly handle the high-frequency, time-series drone telemetry and flight log data required for regulatory compliance (Decree 288).
 - **Redis Stack** operates as a high-speed volatile message queue, acting as a shock absorber to protect physical disk storage from I/O starvation during peak spraying operations [NFR04].
- **Infrastructure & Hosting:** The system is deployed on a single-node VPS/Cloud VM (e.g., DigitalOcean or a local Vietnamese provider such as Viettel IDC/VNG Cloud) rather than a Kubernetes cluster, consistent with the chosen Modular Monolith architecture (Section 4) and the limited DevOps capacity of agricultural cooperatives. [Docker](#) and [Docker Compose](#) package the application, database, and Redis buffer into reproducible containers for straightforward local backup and single-command redeployment. [Nginx](#) serves as the reverse proxy and TLS termination point (detailed further in Section 6). A lightweight [CI/CD pipeline \(GitHub Actions\)](#) automates build, test, and deployment, matching the team's small size (5 students/developers) without requiring dedicated infrastructure staff.

6. DEPLOYMENT TOPOLOGY

This section describes where the components defined in §4 (Architecture) and §5 (Tech Stack) physically run and how they communicate over the network. Business logic (buffering, offline-sync mechanics) is not repeated here see §4.3 for the Redis write-behind pattern and §5 for the SQLite offline-first client design.

6.1 Topology Overview

The AgriUAV Platform uses a **Hybrid Edge Cloud topology**, consisting of two physical zones:

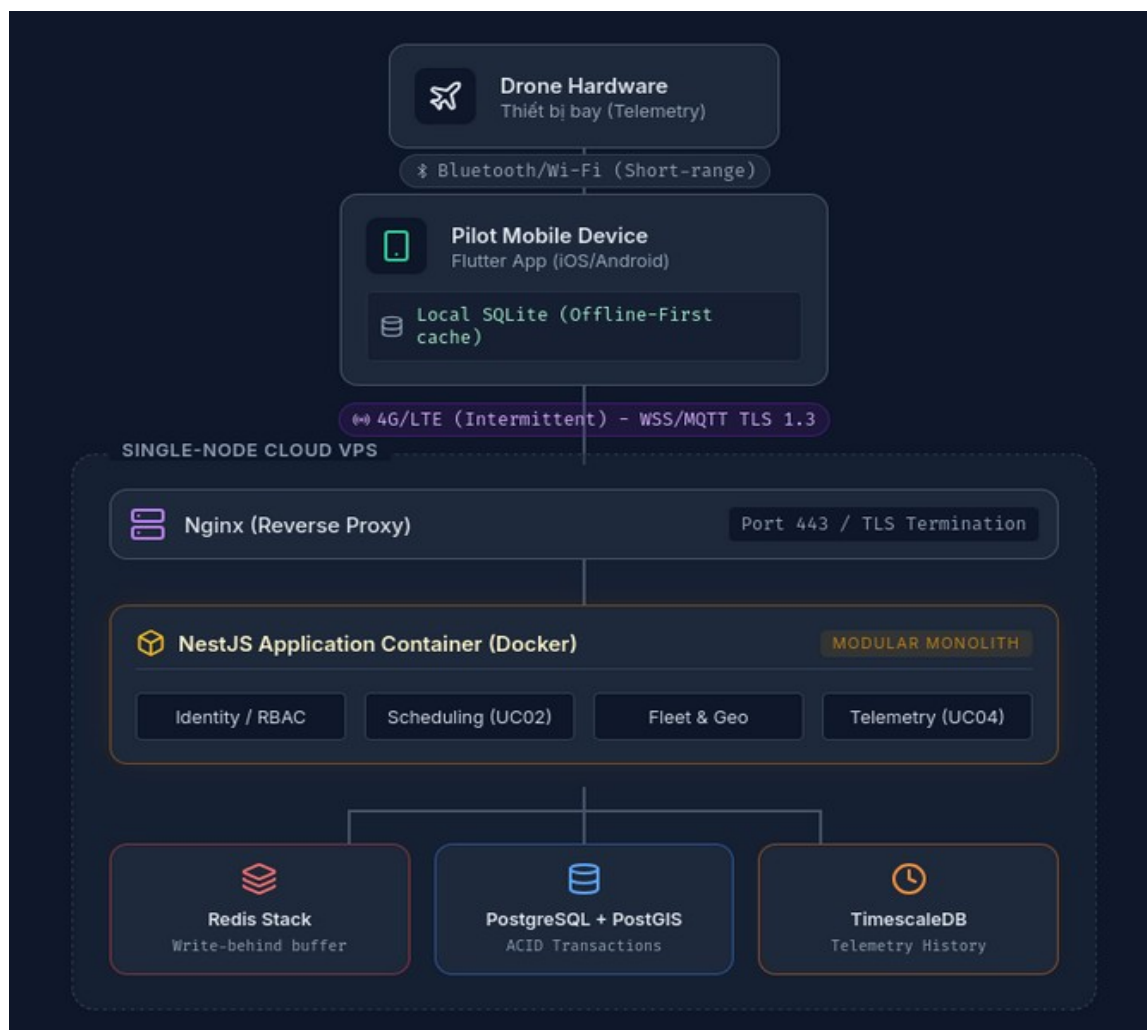
- **Edge Zone** (no infrastructure owned/operated by the team): the Drone hardware and the Pilot's/Farmer's personal smartphone. These devices operate independently of connectivity and only push data upstream opportunistically.
- **Cloud Zone** (single-node VPS, per §5 Infrastructure & Hosting): hosts the entire server-side stack - Nginx, the NestJS application containers, PostgreSQL, TimescaleDB, and Redis — as Docker Compose services on one Vietnamese cloud provider node (e.g., Viettel IDC/VNG Cloud).

There is intentionally no separate edge-compute node (e.g., no local gateway server at the cooperative office) in the MVP scope; all "edge" processing is client-side logic inside the mobile app.

6.2 Physical Network Path

This section illustrates the physical routing of data across the deployed nodes under different operational contexts. The network paths are strictly separated based on payload characteristics: high-throughput IoT streams vs. strict transactional requests.

6.2.1 Baseline Physical Topology

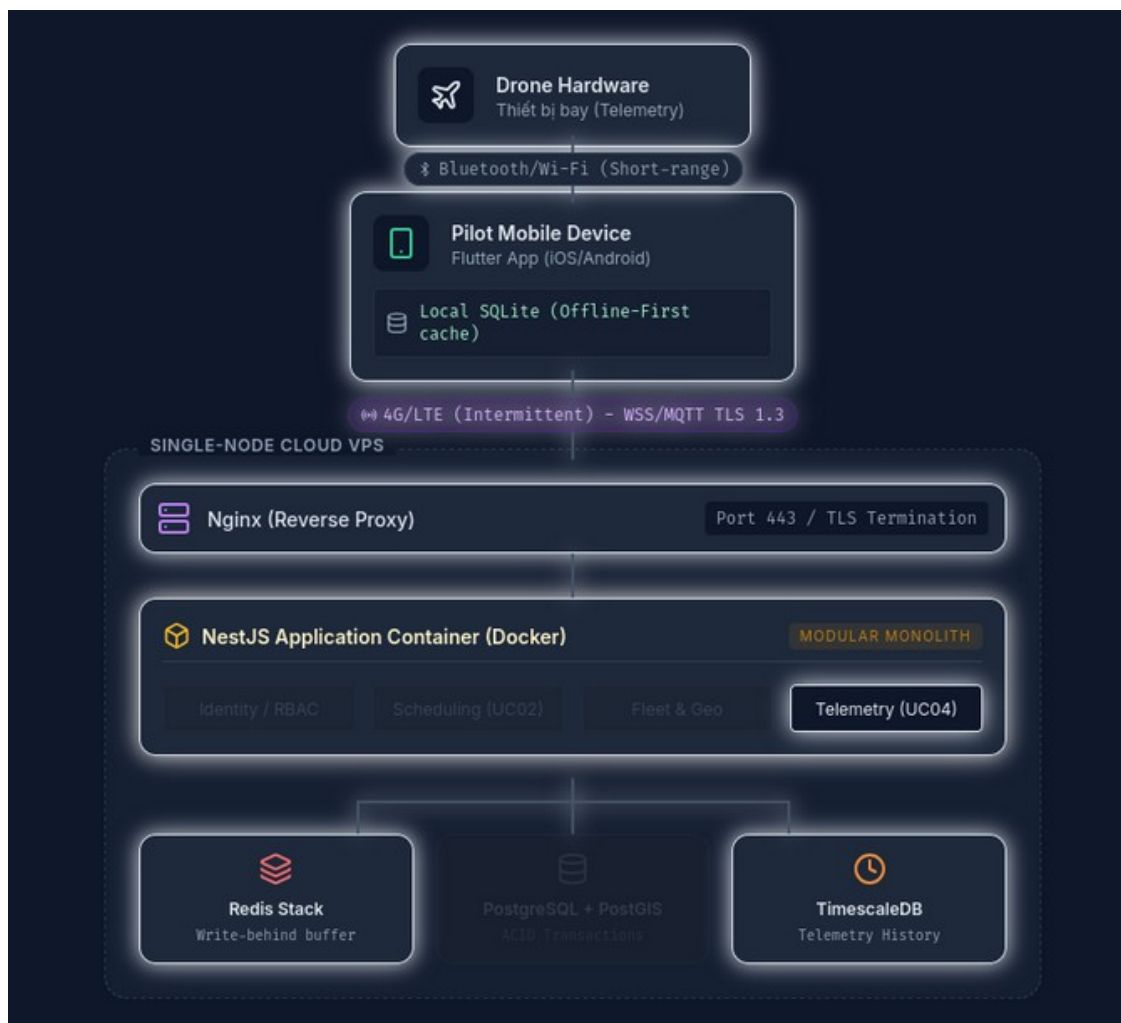


Source: Interactive Architecture Blueprint generated via custom HTML/CSS modeling tool.

Description: The baseline topology consists of three primary physical zones:

- **The Edge (Field Environment):** Drone Hardware communicating via short-range protocols (Bluetooth/Wi-Fi) to the Pilot's Mobile Device.
- **The Gateway (4G/LTE Transport):** The mobile device acts as an edge proxy, handling intermittent 4G connections and buffering data locally via SQLite (NFR06).
- **The Core (Cloud VPS):** A Single-Node containerized environment protected by Nginx (TLS 1.3 Termination), hosting the NestJS Modular Monolith and a polyglot persistence layer (Redis, PostgreSQL, TimescaleDB).

6.2.2 Telemetry Data Path (High-Throughput IoT)

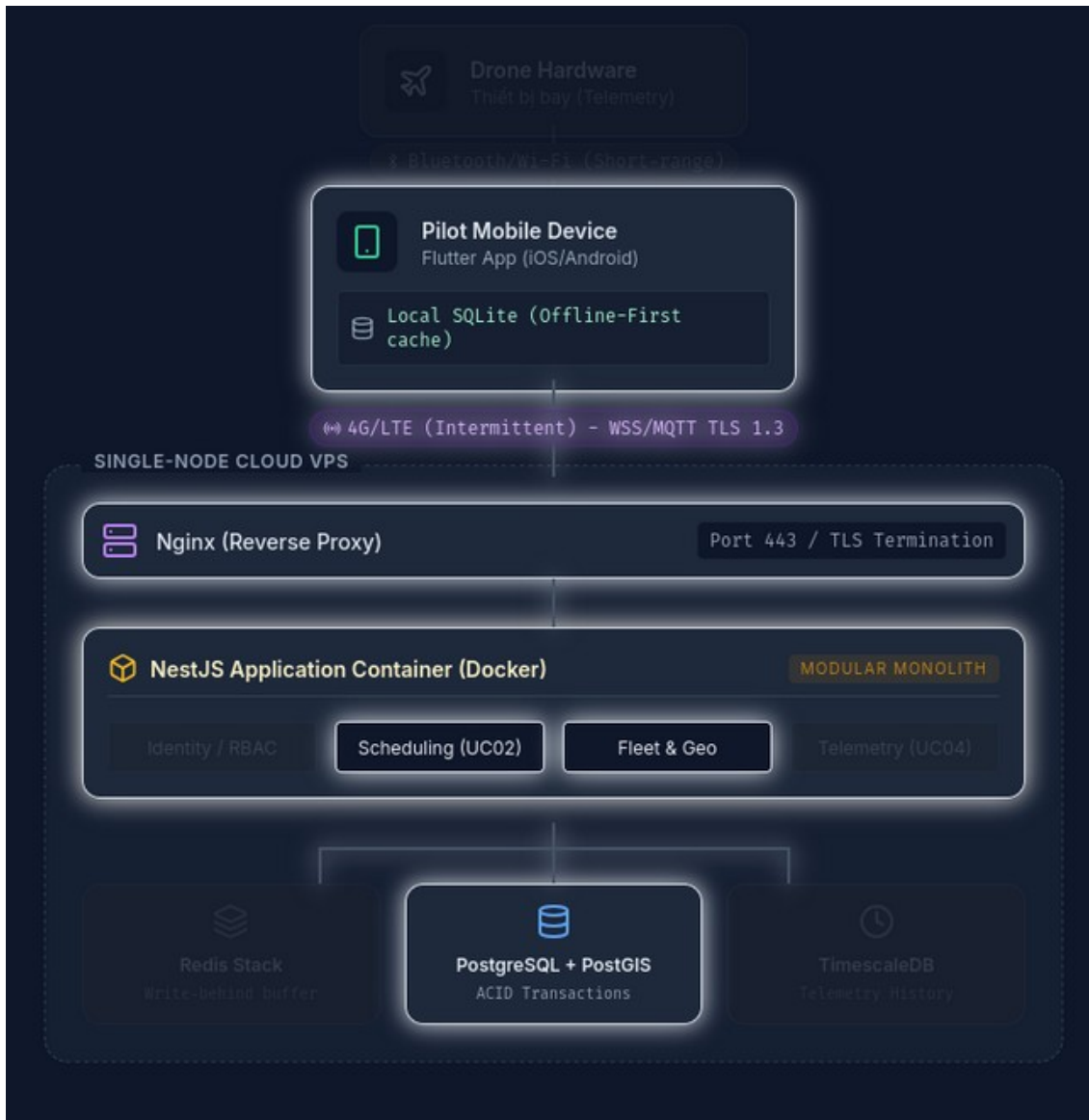


Source: Interactive Architecture Blueprint generated via custom HTML/CSS modeling tool.

Description: This path handles continuous, high-frequency coordinate updates (up to 200 req/s) from the drone fleet.

- **Path:** Drone Hardware → Bluetooth → Mobile App → WSS/MQTT over 4G → Nginx → NestJS (Telemetry Module) → **Redis Stack** → **TimescaleDB**.
- **Justification:** Directing telemetry traffic to the in-memory **Redis Buffer** prevents the primary database from experiencing I/O bottlenecks. A background worker subsequently flushes this data to TimescaleDB for long-term legal compliance storage (Decree 288/2025/ND-CP).

6.2.3 Transactional Booking Path (Strict Consistency)



Source: Interactive Architecture Blueprint generated via custom HTML/CSS modeling tool.

Description: This path processes financial and scheduling requests where data integrity and race-condition prevention are critical.

- **Path:** Mobile App → HTTPS/TLS over 4G → Nginx → NestJS (Booking Module) → PostgreSQL.
- **Justification:** Unlike telemetry, booking requests bypass the Redis buffer and are routed directly to PostgreSQL. This leverages the database's ACID properties and **Row-Level Locking (Pessimistic Lock)** mechanism to instantly block overlapping schedules and prevent double-booking of drone hardware.

6.3 Zone Responsibilities

Zone	Component	Responsibility
Edge	Drone	Captures flight telemetry; has no direct network stack of its own — relays only to the paired mobile device.
Edge	Pilot Mobile Device	Runs the offline-first client (§5); is the only point where a network disconnect is handled — see §5 for the sync mechanism.
Cloud	Nginx	Single entry point; TLS termination; routes to the NestJS container pool.
Cloud	NestJS Containers	Stateless application layer (Booking, Fleet, Telemetry modules per §4.3); horizontally restartable but not horizontally scaled in the MVP (single VPS).
Cloud	PostgreSQL / TimescaleDB / Redis	Co-located on the same VPS in the MVP; see §5 for why (cost, no DevOps team) and §9 for how their health is monitored.

6.4 Backup & Disaster Recovery (Physical Layer)

- Nightly **pg_dump** snapshots of PostgreSQL are stored to a separate object-storage bucket (off-VPS), satisfying the 5-year retention requirement noted in §3 for flight_logs, drone_positions, and audit_logs.
- The single-node VPS is a known single point of failure for the MVP; this is an accepted trade-off consistent with §4.2's Operational Complexity assessment (cooperatives have no dedicated DevOps capacity) and is flagged as a Limitation for the Week 11 defense (§C.1 slide 9, RBL Guide).
- Recovery Time Objective (RTO) target: VPS snapshot restore within the NFR02 ≤ 15 -minute window (D2 §3.2).

6.5 Cross-References

- Redis write-behind buffering mechanism → §4.3
- Offline-first SQLite sync protocol on the Pilot app → §5
- Nginx/TLS role in the request path → §5 (introduced), §6.2 (positioned physically here)
- Monitoring of VPS-level resource thresholds → §9

7. SECURITY & ACCESS CONTROL

7.1 STRIDE Threat Modeling & Mitigation Actions

Threat Category		Target Asset	Identified Vector Detail	Enforced Architectural Mitigation Strategy
Spoofing Identity		PII / Telemetry	An unauthorized operator spoofs a pilot account to submit fake flight completions and claim unearned payments.	Enforce strict JSON Web Tokens (JWT) signed via RSA-256. Combine this with mobile device fingerprinting and require GPS coordinates to match within a 100m radius of the assigned field before allowing check-in.
Tampering with Data		Telemetry / Airspace	An operator alters flight telemetry streams to obscure drone damage or cover up the use of unapproved chemicals.	All data transmissions use mandatory TLS 1.3 transport encryption. Every telemetry payload must include a cryptographic signature generated via HMAC-SHA256 using a rotating key stored securely in the device's keychain.
Repudiation		System Logs	A pilot cancels an assigned spraying mission at the last minute but denies taking the action, leaving the farmer stranded.	The system implements an append-only immutable audit trail module. Any status modifications are automatically stamped with the authenticating user ID, source IP address, and server-side atomic timestamps.
Information Disclosure		PII	Proprietary land boundary shapes, yield metrics, or farmer contact details leak to unauthorized third-party agrochemical businesses.	Sensitive Personal Identifiable Information (PII) columns (e.g., phone numbers, tax identification codes) are encrypted at rest inside PostgreSQL using transparent AES-256 block ciphers.
Denial of Service (DoS)	Telemetry	Malicious actors use a botnet to spam thousands of fake requests per second (exceeding the 200 req/s baseline from NFR04) at the telemetry ingestion port, aiming to sever real-time drone connections.		Enforce Rate Limiting at the API Gateway layer (Nginx). Route all high-frequency telemetry streams through an in-memory Redis Stack Buffer to absorb request spikes before permanently flushing to the database, preventing I/O bottlenecks.
Elevation of Privilege	PII / Airspace	A standard user (Farmer) manipulates API request payloads to grant themselves cooperative administrator (HTX Manager) privileges to bypass payment or approve unverified flights.		Implement strict Role-Based Access Control (RBAC) at the NestJS Guard/Middleware layer. Configure Row-Level Security (RLS) in PostgreSQL to ensure farmers can only access or modify records matching their own userId.

7.2 Role-Based Access Control (RBAC) Permissions Matrix

System Use Case Interface	Farmer Account Role	Pilot (Operator) Account Role	HTX Manager Role	Agronomist Account Role	Admin Account Role
UC01: Create Spray Mission Request	ALLOWED (Own Fields Only)	DENIED	ALLOWED (On behalf of Farmer)	DENIED	ALLOWED (Full Access, cross-cooperative — FR18)
UC02: Dispatch Drone / Assign Pilot	DENIED	DENIED	ALLOWED (Full Fleet Domain)	DENIED	ALLOWED (Full Access, cross-cooperative — FR18)
UC04: Stream Telemetry Flight Log	READ-ONLY (Visual Track)	ALLOWED (Write Active Stream)	READ-ONLY (Fleet Dashboard)	READ-ONLY (Chemical Data)	READ-ONLY (Cross-Cooperative Fleet Oversight)
UC05: Log Emergency Incident Report	DENIED	ALLOWED (Write Incident State)	ALLOWED (Write & Resolve States)	DENIED	ALLOWED (Write & Resolve States, cross-cooperative)
UC09: Configure Chemical Dosage Matrix	DENIED	READ-ONLY (View parameters)	READ-ONLY (View parameters)	ALLOWED (Full Write Access)	READ-ONLY (View Only — not an FR02-authorized confirmer)
UC10: Export Auditable VietGAP Logs	ALLOWED (Own Plots History)	DENIED	ALLOWED (Full Cooperative Export)	ALLOWED (Full Cooperative Export)	ALLOWED (Full Cross-Cooperative Export — Audit Oversight)

Admin's "full access" (FR18) is scoped to cross-cooperative oversight, user/role management, and audit-log access D2 it is not unrestricted override of domain-specific approval authority. UC09 is the deliberate exception: FR02 names only Agronomist or HTX Manager as authorized dosage confirmers, so Admin remains READ-ONLY there even though other rows show full access.

8. QUALITY ASSURANCE (QA) PLAN

8.1 Test Strategy Overview

In alignment with the RBL framework, the testing strategy for the AgriUAV System Design phase spans three complementary test types **unit, integration, and User Acceptance Testing (UAT)** with emphasis shifting toward Architectural Validation and Edge-Case Resilience at the higher levels. Since the core challenges of operating in the Mekong Delta involve unstable 4G networks (NFR06) and strict regulatory auditing (Decree 288/2025/ND-CP), the strategy is layered as follows:

- **Unit Testing:** Isolated validation of core business logic, e.g. the schedule-conflict checker (FR04), the per-zone pesticide dosage calculator (FR02), and the drone/pilot rule-based assignment scorer (FR06), via Jest.
- **Integration Testing:** Validates module-boundary behavior within the Modular Monolith (§4), e.g. Booking Module ↔ PostgreSQL transactional writes, Telemetry Module ↔ Redis write-behind buffer, and NestJS Guard ↔ RBAC enforcement (§7.1).
- **UAT / Architectural Validation:** End-to-end scenarios validating system behavior at its operational boundaries concurrency control, offline resilience, and compliance traceability detailed below.

Testing focus areas include:

- **Concurrency Testing:** Validating PostgreSQL's Row-Level Locking during simultaneous booking requests.
- **Network Throttling/Disconnection:** Using simulated 'Airplane Mode' and background workers to validate the Offline-First mobile architecture (SQLite caching).
- **Data Integrity Verification:** Executing deep backend queries to ensure the unbroken traceability chain of spray-order logs.

8.2 Architectural Test Script Scenarios (Core UC Coverage)

TC_UAT_01: Automated Overlap Prevention Execution (Validating UC02 / Data Concurrency Integrity)

1. Pre-conditions: Inside PostgreSQL, Drone ID 45 is currently flagged as 'IN_FLIGHT' (actively assigned to Spray Order A on 2026-07-20 from 07:00:00+07 to 09:30:00+07).
2. Execution Steps:
 - a. Log in to the Web Portal using an account assigned the 'HTX_MANAGER' role.
 - b. Open the Dispatching Wizard interface and select pending Mission B.
 - c. Attempt to assign Drone ID 45 to Spray Order B on the same date with a schedule from 09:00:00+07 to 11:00:00+07 (creating a 30-minute conflict).
 - d. Click the 'Confirm Assignment' submission interface button.
3. Expected System Behavior: The backend interceptor catches the schedule overlap before executing the write operation. The database transaction rolls back, and the client receives a '409 Conflict' API error code. The user interface displays a clear alert: 'Drone resource unavailable during the requested time window due to a scheduling conflict with an existing order.' No overlapping records are persisted to disk.

TC_UAT_02: Network Loss Resilience Caching and Synchronisation (Validating UC04 / NFR06 – Offline Availability)

1. Pre-conditions: Ensure the mobile application is running on a pilot test device. Turn on 'Airplane Mode' to simulate a total network loss while operating in a remote field.
2. Execution Steps:

- a. Click 'Start Spray Flight Execution' inside the pilot dashboard.
 - b. Generate a simulated flight path covering 5 distinct coordinate points.
 - c. Check the internal application state and inspect the local device storage logs.
 - d. Turn off 'Airplane Mode' to restore a stable 4G network connection.
3. Expected System Behavior: The mobile application remains responsive without lagging or showing crash alerts. The 5 coordinate points are successfully saved to the local client-side SQLite cache (a mirror of the drone_positions/flight_logs structure, extended with a client-only sync flag) with the flag set to is_synced_offline = true. Once the network state changes back to online, a background sync task automatically transmits the cached data array to /api/v1/telemetry/sync. The data is ingested into the central database with zero record loss.

TC_UAT_03: Traceability Chain Validation for Regulatory Compliance (Validating FR13 – Complete Flight Logs, cross-cutting compliance assurance, consistent with the Coverage Type classification in D2 §5.1 / NFR05 – Security & Audit)

1. Pre-conditions: Spray Order ID 88 has an associated flight_logs record with status 'IN_FLIGHT'.
2. Execution Steps:
 - a. Send a critical-failure telemetry payload from the pilot mobile client (e.g. drone malfunction mid-flight), triggering the backend to mark the associated flight_logs record as 'FAILED'.
 - b. Log in as an administrator and query the system database, joining flight_logs → operators (Pilot Certificate Number) and flight_logs → spray_orders → fields → spray_formulas (pesticide dosage data + geospatial boundary).
3. Expected System Behavior: The flight_logs.status for the affected record instantly flips to 'FAILED'; the parent spray_orders record is not silently overwritten but retains its own traceable state. When pulling the generated incident record, the system returns an unbroken data chain: Flight Log ID → Spray Order ID → Pilot License Number (Decree 288 Compliance) → Pesticide Dosage Data → Field Geospatial Boundary. The information chain is 100% complete and immutable (append-only per §7.1 Repudiation control).

(Note: this scenario is intentionally not mapped to a single numbered UC — a catastrophic in-flight failure is not currently modeled as its own UC in D2 (UC05 Alt 4a covers only voluntary early termination by the Pilot, a different scenario). It is treated as validating FR13 as a cross-cutting compliance guarantee, per the same Coverage Type methodology already established in D2 §5.1.)

TC_UAT_04: Role-Based Auditable Export Validation (Validating UC10 / NFR05 - Access Control)

1. Pre-conditions: Inside the database, there are 5 completed flight logs associated with Farmer A's ID, and 3 logs associated with Farmer B's ID.
2. Execution Steps:
 - a. Log in to the Web Portal using Farmer A's account.
 - b. Attempt to generate a VietGAP compliance report spanning the entire cooperative (manipulating the API payload to request all logs).
 - c. Log out, then log in as an account with the 'HTX_MANAGER' role.
 - d. Request the same full cooperative report.
3. Expected System Behavior: For Farmer A, the backend Row-Level Security (RLS) intercepts the unauthorized request and only returns the 5 logs belonging to Farmer A, preventing PII disclosure. For the

HTX_MANAGER, the system successfully aggregates and exports all 8 logs with cryptographic signatures intact, proving the architecture's RBAC matrix operates correctly at the database level.

9. OPERATIONS & MONITORING DESIGN

To ensure continuous availability during peak harvest seasons, the platform includes automated operations and infrastructure monitoring components. Consistent with the "no dedicated DevOps team" premise established in §4.2, all four tools below ship as pre-configured containers inside the same docker-compose.yml defined in §5 there is no manual tuning, no separate monitoring server, and no expectation that HTX staff ever interact with them. Grafana, Winston, and Alertmanager are read-only-by-default dashboards/pipelines watched exclusively by the 5-student development team; this keeps the stack observationally rich without adding operational burden to the cooperative side, preserving the "operational simplicity" argument made in §4.2.

Component	Tool	What It Monitors	NFR / FR Link
Performance Infrastructure Monitoring	Prometheus & Grafana	Host-level CPU utilization, available RAM, open PostgreSQL connections, TimescaleDB chunk compression status, hypertable ingestion rate, and query latency for telemetry reads, and Redis memory usage. Metrics are visualized on a central Grafana dashboard so the dev team can spot bottlenecks before they cause downtime.	NFR02 (Uptime ≥ 99%, RTO ≤ 15 min) early detection of resource exhaustion is the primary lever for hitting the RTO target. NFR01 (position update ≤ 3s / <500ms latency) TimescaleDB query latency is tracked because it is the read path behind the live map.
Structured Log Aggregation	Winston (JSON framework)	All application services emit structured JSON logs classified as INFO (normal events), WARN (non-fatal errors, e.g. a temporary Weather API timeout), or ERROR (critical issues, e.g. database connection failures or storage limits reached).	Directly supports triage of the Weather API dependency noted in D2 §2.5 a WARN-level timeout is exactly the failure mode UC06's Postconditions (Failure) describes ("Weather alert – Unavailable" banner).
Proactive Alerting Pipeline	Alertmanager	Routes webhook alerts to the development team when the PostgreSQL connection pool exceeds 85% capacity, or when backend API error rates exceed 5% within any 60-second window.	NFR04 (Scalability 200 req/s peak telemetry baseline). The 85%/5% thresholds are deliberately conservative headroom markers set below the point where the 200 req/s peak load (§3.2) would saturate the connection pool, giving the team a warning window before the Redis buffer (§4.3) itself starts backing up.

Cross-Reference Note: This section fulfills the monitoring commitment made in §6.3's Zone Responsibilities table "PostgreSQL / TimescaleDB / Redis... see §9 for how their health is monitored" by

explicitly covering all three data-layer components (previously TimescaleDB was omitted; now included above alongside PostgreSQL and Redis).

APPENDIX: DESIGN PHASE AI AUDIT LOG

Entry #21 - W7

Field	Content
DTC Component	Process / Algorithm — Compliance Enforcement
AI Output (tóm tắt)	The AI initially proposed using generic system_logs, with actor_id configured as ON DELETE SET NULL. No database-level constraints were implemented to prevent UPDATE or DELETE operations, relying solely on application-level conventions.
Decision & Refinement	The proposal was rejected because enforcing compliance only at the application layer does not satisfy the immutable audit trail requirement under Decree No. 288/2025/NĐ-CP. Application-layer validation can be bypassed through direct database writes or implementation bugs. The table was renamed to audit_logs, and two database triggers (prevent_audit_log_modification and trg_audit_logs_append_only) were introduced to block UPDATE and DELETE operations at the database engine level. In addition, the foreign key actor_id was changed from ON DELETE SET NULL to ON DELETE RESTRICT to prevent deletion of users with existing audit records. User deactivation is handled through the is_active flag instead of physical deletion.
Verification	Verified against Design Specification 3 (which defines the append-only audit log convention), Section §7.1 (STRIDE – Repudiation), and NFR05.
AI Tool / Prompt	Claude — designed the audit log schema and evaluated whether application-layer enforcement was sufficient or whether database-layer enforcement was required.

Entry #22 - W7

Field	Content
DTC Component	Critical Thinking & Trade-off
AI Output (tóm tắt)	AI analyzed the trade-off matrix, indicating that Microservices have an absolute advantage in Scalability, but warned about high infrastructure operational costs and deployment complexity.
Decision & Refinement	Decided to adopt Layered Architecture. Argued that the actual context at Mekong Delta cooperatives involves low budgets and a lack of dedicated DevOps teams. Microservices would violate the NFR07 commitment (hotfix resolution time ≤ 4 hours). Layered Architecture is fully sufficient to meet the MVP requirements.
Verification	Cross-checked against practical barriers (Cost) and NFR07 (Maintainability). Aligns with the RBL Guide's Justification section.
AI Tool / Prompt	Gemini - "Evaluate and propose the system architecture (Layered Architecture vs. Microservices) based on the 200 req/s load requirement of the AgriUAV project."

Entry #23 - W7

Field	Content
DTC Component	Traceability & Consistency
AI Output (tóm tắt)	AI detected a critical inconsistency: D3 v4.2 utilizes UUID v4, whereas the D4 draft's SQL DDL code uses INT AUTO_INCREMENT for primary keys.

Field	Content
Decision & Refinement	Firmly rejected the current D4 draft and refactored the entire DDL code to strictly use the UUID v4 standard (VARCHAR(36)). UUID is mandatory for resolving distributed data synchronization issues when drones operate in network dead zones (Offline Mode), preventing primary key collisions upon network reconnection.
Verification	Traceability ensures 100% alignment with D3 v4.2 and directly fulfills NFR06 (Availability / Offline Sync).
AI Tool / Prompt	Gemini - "Cross-check data consistency between the Physical Database Schema design draft (D4 v0.51) and the System Models document (D3 v4.2)."

Entry #24 - W7

Field	Content
DTC Component	Critical Thinking & Trade-off
AI Output (tóm tắt)	AI proposed having the API Gateway receive the coordinate data and directly write it to the PostgreSQL/TimescaleDB database to ensure real-time tracking.
Decision & Refinement	Rejected direct database writes. Analyzed that at an operating frequency of 4Hz across 50 drones, the system must process 200 requests/sec, which would cause severe disk I/O bottlenecks. Inserted a Redis In-Memory Buffer layer as a "shock absorber" to cache high-frequency telemetry, utilizing background workers for scheduled asynchronous batch inserts into the long-term database.
Verification	Verified against NFR04 (Scalability threshold: 50 drones/HTX) and ensures compliance with data retention policies under Decree 288/2025/ND-CP.
AI Tool / Prompt	Gemini - "Design the data flow for the Telemetry module (UC04) to continuously ingest flight coordinates transmitted from drone devices to the server."

Entry #25 - W9

Field	Content
DTC Component	Process / Algorithm - Data Sync
AI Output (tóm tắt)	Proposed a simple database dump that uploads the entire local SQLite dataset onto the server upon detecting a network connection. Suggested using standard auto-incrementing integer keys (INT AUTO_INCREMENT) for primary database records to save storage space.
Decision & Refinement	Human Delta: Firmly rejected integer keys. Since the platform operates heavily on an Offline-First model where records are generated on the field without server connection, auto-increment keys would cause massive primary key collisions during sync. Enforced UUID v4 across all entity definitions.
Verification	Global Database Data Dictionary in D4.
AI Tool / Prompt	Claude – "Design the data synchronization mechanism for the pilot's app when transitioning from offline field operations back to online status."